

Nested RAID Levels (সহজ ব্যাখ্যা)

সহজ করে বুঝি

ধরো, RAID-0, RAID-1, RAID-5 এগুলো প্রত্যেকটি আলাদা আলাদা RAID।

কিন্তু যদি দুই বা তার বেশি RAID একসাথে ব্যবহার করা হয়, তখন সেটাকে Nested RAID (Hybrid RAID) বলে।

☞ অর্থাৎ,

Nested RAID = দুই বা তার বেশি Standard RAID-এর Combination

এতে তিনটি সুবিধা একসাথে পাওয়া যায়—

- বেশি Speed
- বেশি Data Safety
- ভালো Performance

এজন্য বর্তমানে Nested RAID অনেক বেশি জনপ্রিয়।

মনে রাখবে

Standard RAID

- RAID-0
- RAID-1
- RAID-5

Nested RAID

- RAID-10
- RAID-01
- RAID-50
- RAID-60
- RAID-100

Exam Ready Answer (English)

Nested RAID Levels

Definition:

Nested RAID (Hybrid RAID) is a RAID configuration that combines two or more standard RAID levels into one system to achieve better performance, higher fault tolerance, and improved data protection.

Key Points

- Combines multiple standard RAID levels.
- Also known as Hybrid RAID.
- Uses striping, mirroring, and/or parity together.
- Provides better speed and reliability than a single RAID level.
- Examples: RAID-10, RAID-01, RAID-50, RAID-60, RAID-100.

RAID-10 (RAID-1+0)

সহজ করে বুঝি

RAID-10 = RAID-1 + RAID-0

প্রথমে **Mirror (RAID-1)** করা হয়।

তারপর সেই Mirror-এর উপর **Striping (RAID-0)** করা হয়।

অর্থাৎ

☞ **Mirror First → Striping Later**

এজন্য এটাকে **RAID-1+0** বলা হয়।

কিভাবে কাজ করে?

ধরো ৪টি ডিস্ক আছে।

Disk-1 ↔ Disk-2 (Mirror)

Disk-3 ↔ Disk-4 (Mirror)

এখন এই দুই Mirror Pair-এর মধ্যে Striping করা হয়।

Mirror Pair 1 Mirror Pair 2

D1 <--> D2 D3 <--> D4

\ /
 \ /
 Striping

বৈশিষ্ট্য (বাংলায়)

- RAID-1 এবং RAID-0 এর Combination।
- RAID-1 এর চেয়ে অনেক Fast।
- সব Data Mirror হওয়ায় নিরাপদ।
- প্রতিটি Mirror Pair থেকে ১টি করে Disk নষ্ট হলেও কাজ চলবে।
- Minimum 4 Disk লাগে।
- মোট Storage-এর অর্ধেক ব্যবহার করা যায়।

উদাহরণ

৪টি × ১TB

= মোট ৪TB

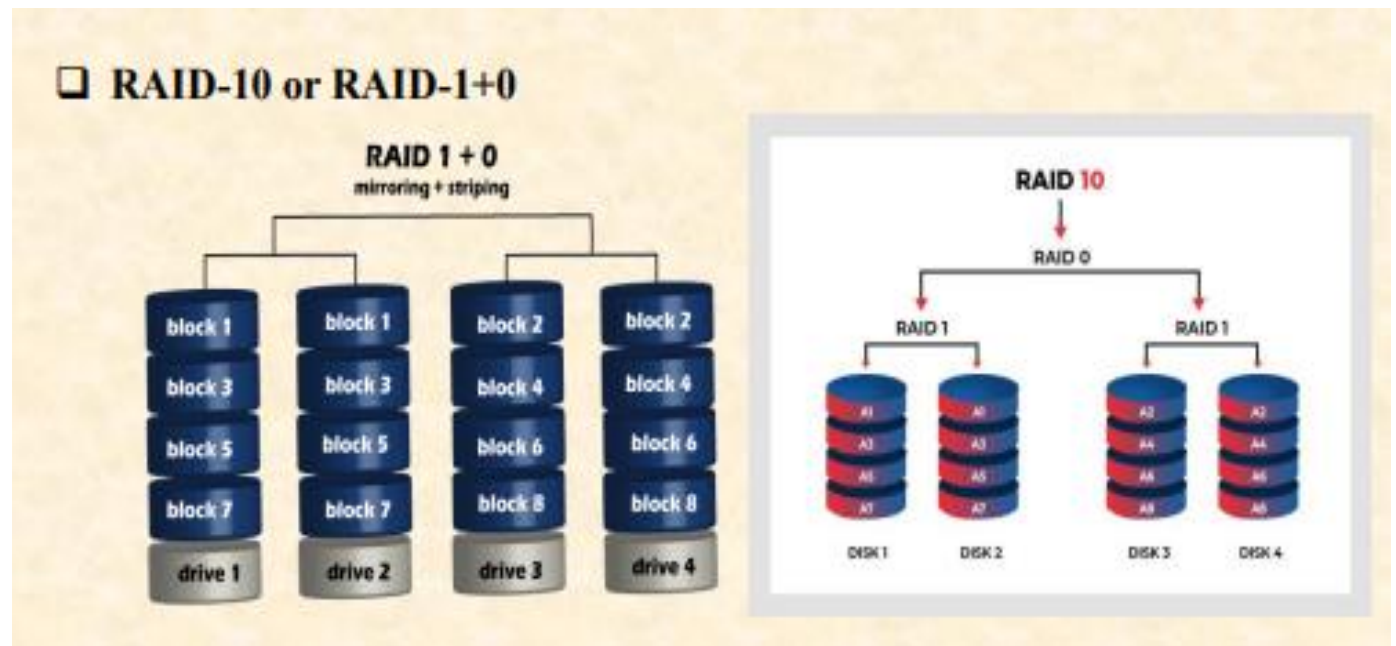
কিন্তু ব্যবহার করা যাবে

= ২TB

কারণ অর্ধেক Mirror-এর জন্য ব্যবহৃত হয়।

Exam Ready Answer (English)

RAID-10 (RAID-1+0)



Definition:

RAID-10 is a nested RAID level that combines RAID-1 (Mirroring) and RAID-0 (Striping). It first mirrors the data and then performs striping across the mirrored sets.

Key Features

- Combines RAID-1 and RAID-0.
- Also called RAID-1+0 or Mirrored RAID-0.
- Provides high performance and high fault tolerance.
- Can survive up to two disk failures (one disk from each mirror pair).
- Requires a minimum of 4 disks.
- Usable storage is 50% of the total capacity due to mirroring.

RAID-01 (RAID-0+1)

সহজ করে বুঝি

RAID-01 = RAID-0 + RAID-1

এখানে RAID-10 এর উল্টো কাজ হয়।

প্রথমে Striping করা হয়।

তারপর পুরো Stripe Set-এর Mirror তৈরি করা হয়।

অর্থাৎ

🔗 **Striping First → Mirroring Later**

তাই একে RAID-0+1 বলা হয়।

কিভাবে কাজ করে?

Stripe Set 1	Stripe Set 2
D1 ----- D2	D3 ----- D4
Mirror	

RAID-10 আর RAID-01 এর পার্থক্য

RAID-10

→ আগে Mirror

→ পরে Striping

RAID-01

→ আগে Striping

→ পরে Mirror

এটাই সবচেয়ে গুরুত্বপূর্ণ পার্থক্য।

বৈশিষ্ট্য (বাংলায়)

- RAID-0 এবং RAID-1 এর Combination।
- আগে Striping করা হয়।
- পরে পুরো Stripe Set Mirror করা হয়।
- Usable Storage RAID-10 এর মতোই।
- Minimum 4 Disk লাগে।

Exam Ready Answer (English)

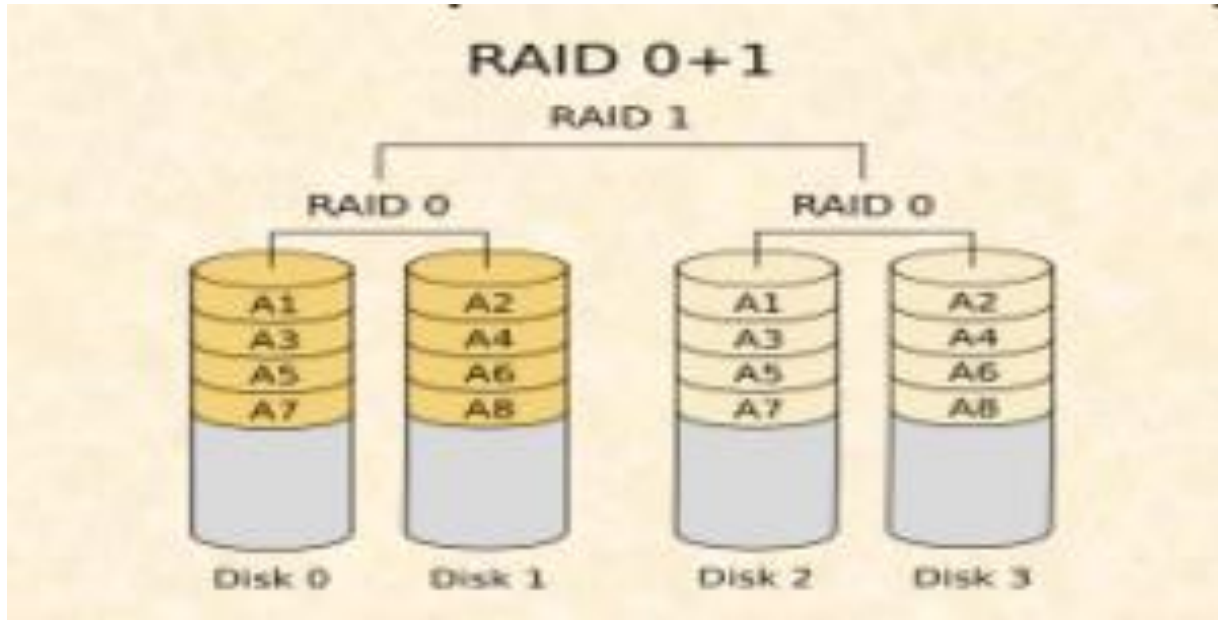
RAID-01 (RAID-0+1)

Definition:

RAID-01 is a nested RAID configuration that combines RAID-0 and RAID-1. It first creates a striped set and then mirrors the entire striped set.

Key Features

- Combines RAID-0 and RAID-1.
- Also called RAID-0+1.
- Performs striping first and mirroring later.
- Provides improved performance and data protection.
- Requires at least 4 disks.
- Usable storage is 50% of the total capacity.



RAID-03 (RAID-0+3)

সহজ করে বুঝি

RAID-03 = RAID-0 + RAID-3

এখানে

RAID-0 এর মতো Striping করা হয়।

তারপর RAID-3 এর মতো Dedicated Parity ব্যবহার করা হয়।

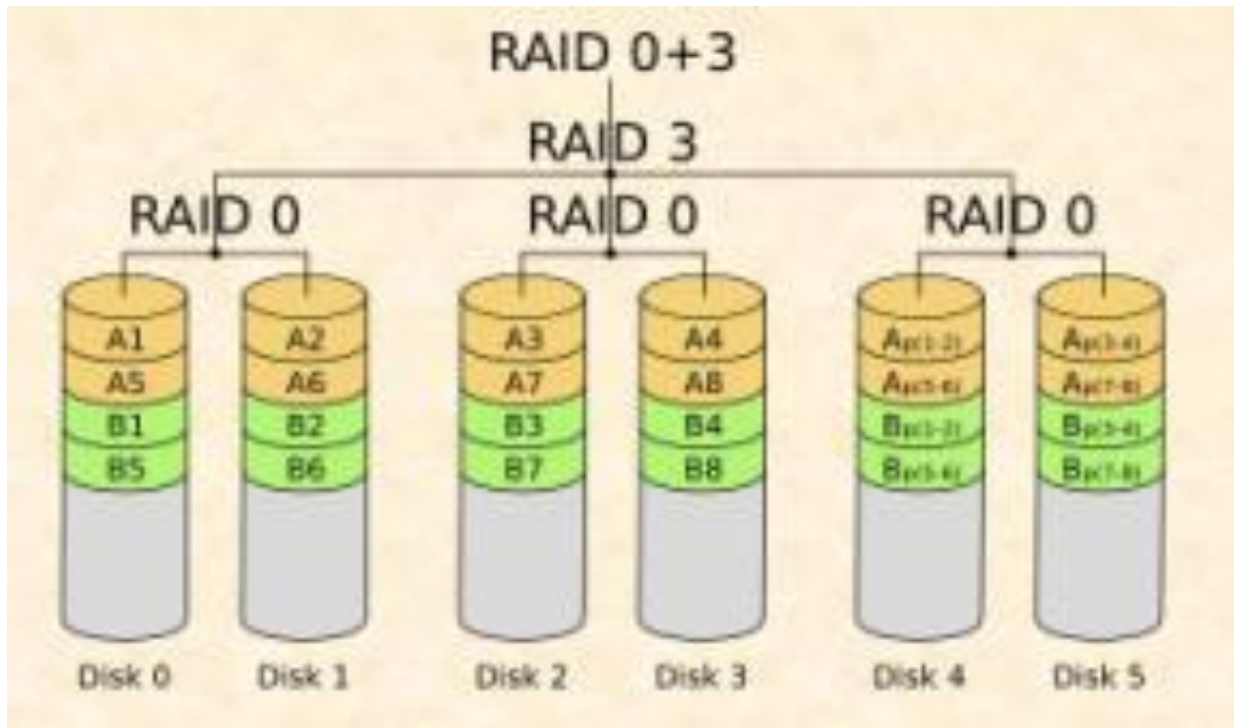
Mirror ব্যবহার করা হয় না।

বৈশিষ্ট্য (বাংলায়)

- RAID-0 এবং RAID-3 এর Combination।
- Striping ব্যবহার করে Speed বাড়ায়।
- Dedicated Parity ব্যবহার করে Data Protection দেয়।
- RAID-3 এর চেয়ে Performance বেশি।
- Cost-ও বেশি।

Exam Ready Answer (English)

RAID-03 (RAID-0+3)



Definition:

RAID-03 is a nested RAID level that combines RAID-0 and RAID-3. It uses striping for high performance and dedicated parity for data protection.

Key Features

- Also called RAID-53.
- Combines RAID-0 and RAID-3.
- Uses striping with dedicated parity.
- Provides better performance than RAID-3.
- More expensive than RAID-3.

★ Exam Trick (১ মিনিটে মনে রাখার উপায়)

RAID Level	সহজে মনে রাখো	কাজের ক্রম
RAID-10 (1+0) Mirror → Striping	আগে Mirror, পরে Striping	
RAID-01 (0+1) Striping → Mirror	আগে Striping, পরে Mirror	
RAID-03 (0+3) Striping + Parity	Striping + Dedicated Parity (Mirror নেই)	

খুব গুরুত্বপূর্ণ (Exam Point)

- Nested RAID = Combination of two or more standard RAID levels.
- RAID-10 = Mirror first, then Striping.
- RAID-01 = Striping first, then Mirror.
- RAID-03 = Striping + Dedicated Parity (No Mirroring).

এই ৪টি লাইন মনে রাখতে পারলে Nested RAID থেকে সাধারণত যে কোনো ৫ নম্বর প্রশ্ন খুব ভালোভাবে লেখা যাবে।

Nested RAID Levels → Slide theke study korbo

মূল কনসেপ্ট: লগ (Log) এবং ডেফার্ড আপডেট (Deferred Update)

ধরে নাও, তুমি একটি ব্যাঙ্কে কাজ করো। মূল সার্ভারে (Database) কোনো কিছু চূড়ান্তভাবে আপডেট করার আগে তুমি একটি সুরক্ষিত রাফ খাতায় (Log) সব হিসাব লিখে রাখো।

১. Log-Based Recovery (রাফ খাতায় বা লগে হিসাব রাখা): ডেটাবেসে কোনো ট্রানজেকশন বা পরিবর্তন করার আগে, সেই পরিবর্তনের সব রেকর্ড একটি সুরক্ষিত জায়গায় (Log) লিখে রাখাকেই Log-Based Recovery বলে।

- এই লগে লেখা থাকে: কে ট্রানজেকশন করছে (Transaction ID), কোথায় করছে, ডেটার আগের ভ্যালু (Old value) কত ছিল, নতুন ভ্যালু (New value) কত হবে, কখন কাজ শুরু হলো (Start) এবং কখন সফলভাবে শেষ হলো (Commit)।

২. Deferred Database Modification (সব শেষে মূল ডেটাবেস আপডেট করা): Deferred মানে হলো দেরি করা। এর মূল নিয়ম হলো— "পুরো কাজ সফলভাবে শেষ (Commit) না হওয়া পর্যন্ত মূল ডেটাবেসে কোনো পরিবর্তন করা যাবে না।" অর্থাৎ, ট্রানজেকশন চলাকালীন সব হিসাব-নিকাশ শুধু লগে সেভ হবে। যখনই ট্রানজেকশনটি "Commit" (চূড়ান্ত) হবে, ঠিক তখনই সেই নতুন ভ্যালুগুলো লগ থেকে নিয়ে মূল ডেটাবেসে আপডেট করা হবে।

৩. ক্র্যাশ করলে কীভাবে ডেটা রিকভারি হয়? (Failure Scenarios): যেহেতু আমরা কাজ পুরোপুরি শেষ না হওয়া পর্যন্ত মূল ডেটাবেসে হাত দিই না, তাই সিস্টেম ক্র্যাশ করলে রিকভারি করা খুব সহজ:

- **Condition 1 (Commit করার আগেই ক্র্যাশ):** ধরো, কাজ চলছিল কিন্তু শেষ (Commit) হওয়ার আগেই সিস্টেম ক্র্যাশ করলো। যেহেতু মূল ডেটাবেসে এখনো কোনো আপডেট করা হয়নি, তাই সিস্টেম ঠিক হওয়ার পর তোমাকে কিছুই করতে হবে না (No Redo)। আগের ডেটা আগের মতোই থাকবে।
- **Condition 2 (একজনের Commit হয়েছে, অন্যজনের হয়নি):** ধরো, T1 এবং T2 দুটি কাজ চলছিল। T1 কাজ শেষ করে Commit করেছে, কিন্তু T2 শেষ করার আগেই ক্র্যাশ হলো। তখন সিস্টেম চালু হওয়ার পর শুধু T1-এর নতুন ভ্যালুগুলো মূল ডেটাবেসে আপডেট (Redo) করে দেওয়া হবে। আর T2 বাতিল করে আবার নতুন করে শুরু (Re-execute) করতে হবে।

- **Condition 3 (সবার Commit হওয়ার পর ক্র্যাশ):** যদি T1 এবং T2 দুটোর কাজই Commit হয়ে যাওয়ার পর সিস্টেম ক্র্যাশ করে, তবে সিস্টেম চালু হওয়ার পর দুটোর ডেটাই লগ দেখে দেখে মূল ডেটাবেসে আপডেট (Redo) করে দেওয়া হবে।

💡 **এক লাইনে মূল কথা:** লগের মধ্যে যদি Start এবং Commit দুটোই লেখা থাকে, তবেই শুধু কাজটা মূল ডেটাবেসে আপডেট (Redo) করতে হবে। আর যদি শুধু Start লেখা থাকে কিন্তু Commit না থাকে, তবে কিছুই করার দরকার নেই!

1. Log-Based Recovery Fundamentals

- **What is Log-Based Recovery?**
 - The log is a sequence of records maintained in stable storage to recover data in case of failure.
 - The process of storing logs should be done before the actual transaction is applied in the database.
- **Fields of a Log Record:**
 - **Transaction identifier:** Unique identifier distinguishing one transaction from another.
 - **Data-item identifier:** Uniquely identifies the data's location on the disk.
 - **Old value:** The value of the data item before writing.
 - **New value:** The value of the data item after it has been written.
- **Types of Log Records:**
 - **<Ti start>:** Transaction Ti has started.
 - **<Ti, V1, V2 Xj, >:** Ti performed a write operation on Xj, changing it from V1 to V2.
 - **<Ti commit>:** Transaction Ti has committed.
 - **<Ti abort>:** Transaction Ti has aborted.

❑ Types of Log Records

- There are various types of log records:

Log Records	Description
<Ti start>	Transaction Ti has started.
<Ti, Xj, V1, V2>	Transaction Ti has performed a write operation on data item Xj. Xj contains value V1 before the write and will contain value V2 after the write.
<Ti commit>	Transaction Ti has committed.
<Ti abort>	Transaction Ti has aborted.

- Whenever a transaction performs a write, a log record is created before the database is modified.
- The logs may or may not record the reading of data items. Because reading a data item has no effect on the consistency of the database.

Recovery Operations:

Database crash হলে বা transaction মাঝপথে বন্ধ হয়ে গেলে **Recovery Operation** ব্যবহার করে database-কে সঠিক অবস্থায় ফিরিয়ে আনা হয়।

1. Undo (Ti) = আগের অবস্থায় ফিরিয়ে নেওয়া (Rollback)

- যদি Transaction সম্পূর্ণ না হয় (Uncommitted), তাহলে তার করা সব পরিবর্তন বাতিল করা হয়।
- অর্থাৎ Old Value আবার ফিরিয়ে আনা হয়।

Example:

- আগে Balance = 1000 টাকা
- Transaction আপডেট করে = 800 টাকা
- কিন্তু Commit হওয়ার আগেই System Crash হলো।

☞ Undo করলে আবার Balance = 1000 টাকা হবে।

সহজে মনে রাখো:

Undo = ভুল হলে আগের অবস্থায় ফিরে যাও।

2. Redo (Ti) = নতুন পরিবর্তন আবার প্রয়োগ করা

- যদি Transaction Commit হয়ে যায়, কিন্তু Crash-এর কারণে পরিবর্তন ডিস্কে পুরোপুরি সংরক্ষিত না হয়, তাহলে সেই New Value আবার লেখা হয়।

Example:

- আগে Balance = 1000 টাকা
- Transaction Commit করে Balance = 800 টাকা
- এরপর System Crash হলো।

☞ Redo করলে আবার Balance = 800 টাকা লেখা হবে।

সহজে মনে রাখো:

Redo = Commit হওয়া কাজ আবার করে দাও।

Exam Ready (English)

- **Undo (Ti):** Restores the **old values** of data items if transaction **Ti** did not commit.
- **Redo (Ti):** Reapplies the **new values** of data items if transaction **Ti** had already committed.

এক লাইনে মনে রাখার ট্রিক

- Undo = Old Value (Rollback)
- Redo = New Value (Repeat Committed Changes)

Database Modification Techniques for ensuring transaction atomicity

There are two approaches to modify the database:

- Deferred database modification and
- Immediate database modification

Deferred Database Modification Technique (বাংলা)

Deferred Database Modification হলো এমন একটি Recovery Technique যেখানে **Transaction Commit না হওয়া পর্যন্ত Database Update করা হয় না**। প্রথমে সব পরিবর্তন **Log File**-এ সংরক্ষণ করা হয়। Commit হওয়ার পরে Database Update করা হয়। এজন্য একে **NO-UNDO / REDO Technique** বলা হয়।

উদাহরণ

প্রাথমিকভাবে Database-এর অবস্থা ছিল—

- A = 1000
- B = 1500
- C = 2000

Transaction T1

A থেকে ২০০ টাকা কেটে B-তে যোগ করা হয়।

- A = 1000 – 200 = **800**
- B = 1500 + 200 = **1700**

Transaction T2

C থেকে ১০০ টাকা উত্তোলন করা হয়।

- C = 2000 – 100 = **1900**

T1	T2
Read (A) A := A – 200	Read (C) C := C – 100
Write (A) Read (B) B := B + 200	Write (C)
Write (B)	

Log File:

Log	Database
<T1 start> <T1, A, 800> <T1, B, 1700> <T1 commit>	A=800 B=1700
<T2 start> <T2, C, 1900> <T2 commit>	C=1900

Condition 1	Condition 2	Condition 3
<T1 start> <T1, A, 800> <T1, B, 1700>	<T1 start> <T1, A, 800> <T1, B, 1700> <T1 commit> <T2 start> <T2, C, 1900>	<T1 start> <T1, A, 800> <T1, B, 1700> <T1 commit> <T2 start> <T2, C, 1900> <T2 commit>
Crash	Crash	Crash

Recovery-এর তিনটি অবস্থা

Condition 1

Crash হওয়ার আগে T1 Commit হয়নি।

Recovery:

- কোনো Redo হবে না।
- কারণ Commit না হওয়া পর্যন্ত Database Update হয় না।

Database:

- A = 1000
- B = 1500
- C = 2000

Condition 2

Crash হওয়ার আগে T1 Commit হয়েছে, কিন্তু T2 Commit হয়নি।

Recovery:

- Redo(T1) করা হবে।
- T2 need No Undo ;No Undo(T2)

Database:

- A = 800
- B = 1700
- C = 2000

Condition 3

Crash হওয়ার আগে T1 ও T2 উভয়ই Commit হয়েছে।

Recovery:

- Redo(T1)
- Redo(T2)

Database:

- A = 800
- B = 1700
- C = 1900

Immediate Database Modification Technique (Exam Ready - বাংলা)

Immediate Database Modification হলো এমন একটি Recovery Technique যেখানে Transaction চলাকালীনই Database Update করা হয়। Log File-এ প্রতিটি Update-এর Old Value এবং New Value সংরক্ষণ করা হয়। তাই একে UNDO/REDO Technique বলা হয়।

উদাহরণ

প্রাথমিকভাবে Database-এর অবস্থা ছিল—

- A = 1000
- B = 1500
- C = 2000

Transaction T1

A থেকে ২০০ টাকা কেটে B-তে যোগ করা হয়।

- A = 1000 → 800
- B = 1500 → 1700

Transaction T2

C থেকে ১০০ টাকা উত্তোলন করা হয়।

- C = 2000 → 1900

T1	T2
Read (A) A := A - 200 Write (A) Read (B) B := B + 200 Write (B)	Read (C) C := C - 100 Write (C)

Log	Database
<T1 start> <T1, A, 1000, 800> <T1, B, 1500, 1700> <T1 commit>	A=800 B=1700
<T2 start> <T2, C, 2000, 1900> <T2 commit>	C=1900

Condition 1	Condition 2	Condition 3
<T1 start> <T1, A, 1000, 800> <T1, B, 1500, 1700>	<T1 start> <T1, A, 1000, 800> <T1, B, 1500, 1700> <T1 commit> <T2 start> <T2, C, 2000, 1900>	<T1 start> <T1, A, 1000, 800> <T1, B, 1500, 1700> <T1 commit> <T2 start> <T2, C, 2000, 1900> <T2 commit>
Crash	Crash	Crash

এখানে প্রতিটি Log-এ Old Value এবং New Value উভয়ই সংরক্ষিত থাকে।

Recovery-এর তিনটি অবস্থা

Condition 1

Crash হওয়ার আগে T1 Commit হয়নি।

Recovery:

- Undo(T1) করা হবে।
- কারণ Database আগে থেকেই Update হয়েছে কিন্তু Commit হয়নি।

Final Database:

- A = 1000
- B = 1500
- C = 2000

Condition 2

Crash হওয়ার আগে T1 Commit হয়েছে, কিন্তু T2 Commit হয়নি।

Recovery:

- Redo(T1)
- Undo(T2)

Final Database:

- A = 800
- B = 1700
- C = 2000

Condition 3

Crash হওয়ার আগে T1 ও T2 উভয়ই Commit হয়েছে।

Recovery:

- Redo(T1)
- Redo(T2)

Final Database:

- A = 800
- B = 1700
- C = 1900

4. Differences between Deferred Update and Immediate Update

Deferred Update:

- It is also called the NO-UNDO/REDO technique.
- Whenever any transaction is executed, the updates are not made immediately to the database; they are first recorded on the log file and then those changes are applied once the commit is done (this is called the "Re-doing" process).
- Once the rollback is done, none of the changes are applied to the database and the changes in the log file are also discarded.

Immediate Update:

- It is also called the UNDO/REDO technique.
- Whenever any transaction is executed, the updates are made directly to the database and the log file is also maintained which contains both old and new values.
- Once the commit is done, all the changes get stored permanently in the database, and records in the log file are thus discarded.
- Once rollback is done the old values get restored in the database and all the changes made to the database are also discarded (this is called the "Un-doing" process).

পরীক্ষার জন্য **Checkpoints in DBMS** এর কনসেপ্টটা একদম সহজ করে একটি বাস্তব উদাহরণের সাহায্যে নিচে বুঝিয়ে দেওয়া হলো। এরপর স্লাইডগুলো থেকে ফুল মার্কস পাওয়ার মতো গুছানো Exam-Ready Answer তৈরি করে দিয়েছি।

□ মূল কনসেপ্ট (সহজ বাস্তব উদাহরণ):

ধরো, তুমি একটি অনেক বড় ভিডিও গেম খেলছ। গেমের যদি কোনো "Save Point" না থাকে, তবে গেম ওভার হলে তোমাকে একদম লেভেল ১ থেকে আবার শুরু করতে হবে, যা অনেক বিরক্তিকর এবং এতে প্রচুর সময় নষ্ট হয়।

ডেটাবেসের ক্ষেত্রেও ঠিক একই ঘটনা ঘটে। প্রতিদিন হাজার হাজার ট্রানজেকশন হয়। সিস্টেম হঠাৎ ক্র্যাশ করলে একদম প্রথম থেকে সব লগ চেক করে রিকভারি করা প্রায় অসম্ভব। তাই ডেটাবেস কিছুক্ষণ পরপর একটি "Save Point" বা **Checkpoint** তৈরি করে রাখে, যেখানে সব কাজ পারফেক্টলি সেভ করা থাকে। ক্র্যাশ করলে ডেটাবেস একদম প্রথম থেকে শুরু না করে, শুধু শেষ Checkpoint থেকে রিকভারি শুরু করে। এতে রিকভারি অনেক দ্রুত হয়।

📌 Exam-Ready Notes: Checkpoints in DBMS

1. What is a Checkpoint?

- A Checkpoint is used to declare a point before which the DBMS was in a consistent state, and all transactions were committed.
- During transaction execution, checkpoints are traced, and upon reaching a save point, the log file is destroyed by saving its update to the database.
- A new log is then created with upcoming execution operations of the transaction and it will be updated until the next checkpoint and the process continues.

2. Why do we need Checkpoints?

- Searching the entire log to determine which transactions need to be redone and undone in case of system failures is very time-consuming.
- Most transactions which need to be redone have already written their updates to the database.
- For example, in an online database system like an airline reservation system handling hundreds of transactions per minute, the log contains a massive volume of information.
- Checkpoints are utilized for removing all previous transaction logs and storing them in permanent storage, establishing a scheme to limit the log information that has to be handled and processed in case of a system failure.
- When a checkpoint occurs, the DBMS will write a copy of the current state of the database to disk.

3. Types of Checkpoints

- **Automatic Checkpoint:** These occur at regular intervals (e.g., every hour or day), are set by the database administrator, and are generally used by heavy databases to recover data easily. For example, in SQL Server, the default interval for automatic checkpoints is every minute.
- **Manual Checkpoint:** These are manually set by the database administrator, are generally used for smaller databases, and are updated very less frequently.

4. Operations Performed When a System Issues a Checkpoint

- The log information from the buffers in the volatile storage is copied to the log on the stable storage.
- All database updates from the buffers in the volatile storage are written to the disk.
- A checkpoint record is written to log on the disk, so subsequent database recovery coordinates with the checkpoint.

5. Advantages, Disadvantages & Applications

- **Advantages:** Checkpoints help in recovering the transaction of the database in case of failure, enhance the consistency of the database when multiple transactions execute simultaneously, prevent unnecessary redo operations, and have a very low overhead allowing them to be done frequently.
- **Disadvantages:** They can only be used to restore from logical errors and cannot be used to restore files due to a media failure because all data blocks are on the same physical device.
- **Applications:** Primarily used for **Backup and Recovery** (ensuring quick recovery and no data loss), **Performance Optimization** (reducing the amount of work needed during recovery), and **Auditing** (tracking changes to the database over time).

6. Recovery Using Checkpoints

- The recovery system reads the logs backward from the end to the last checkpoint.
- It maintains two lists – an undo-list and a redo-list.
- Whenever the recovery system sees a log with `<Tn, start>` and `<Tn, commit>` or only `<Tn, commit>`, it will put that transaction in the redo-list (e.g., T1, T2, and T3).
- Whenever the recovery system sees a log with `<Tn, start>` but no commit, it will put that transaction in the undo-list (e.g., T4).
- All the transactions in the redo-list are deleted with their previous logs and then redone before saving their logs.
- All the transactions in the undo-list are undone and their logs are deleted.